

# Advanced Statistical Learning

## Kernel Smoothing Methods

Xiaolei Lin

School of Data Science  
Fudan University

Nov 3, 2025

# Outline

1-Dim Kernel Smoother

Local Regression in  $\mathbb{R}^p$

Structured Local Regression Models in  $\mathbb{R}^p$

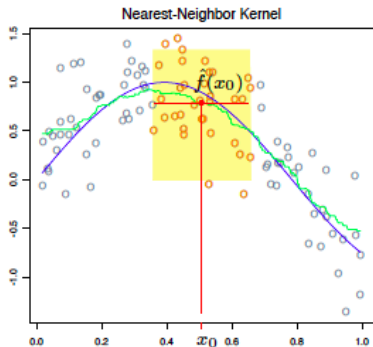
Local likelihood & Other Models

Kernel Density Estimation & Classification

Radial Basis Function & Kernels

Mixture Models for Density Estimation & Classification

# KNN



Blue: true function  
Green: KNN smoother

k-nearest-neighbour estimate:

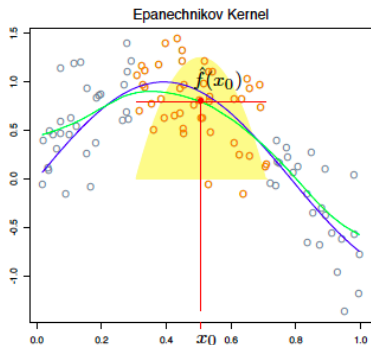
$$\hat{f}(x) = \text{Ave}(y_i \mid x_i \in N_k(x))$$

$N_k(x)$ : the set of  $k$  points nearest to  $x$  in squared distance

$\hat{f}(x)$ : bumpy, discontinuous in  $x$

- : fitted constant  $\hat{f}(x_0)$
- : observations contributing to the fit at  $x_0$
- region: weights assigned to observations

# Kernel Weighted - Epanechnikov



Blue: true function  
Green: Kernel smoother

Kernel weighted average:

$$\hat{f}(x_0) = \frac{\sum_{i=1}^N K_\lambda(x_0, x_i) y_i}{\sum_{i=1}^N K_\lambda(x_0, x_i)}$$

with the Epanechnikov quadratic kernel

$$K_\lambda(x_0, x) = D\left(\frac{|x - x_0|}{\lambda}\right)$$

with

$$D(t) = \begin{cases} \frac{3}{4}(1 - t^2), & \text{if } |t| \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

# Kernel Weighted - Varying Metric Window Size

The previous Epanechnikov kernel weighted average

- ▶ constant metric window size  $\lambda = 0.2$
- ▶ does not change while  $x_0$  moves
- ▶ KNN adapts to the local density of  $x_i$

Kernel with adaptive neighborhood

$$K_{\lambda}(x_0, x) = D\left(\frac{|x - x_0|}{h_{\lambda}(x_0)}\right)$$

- ▶ For KNN,  $h_k(x_0) = |x_0 - x_{[k]}|$
- ▶ where  $x_{[k]}$  is the  $k_{th}$  closest  $x_i$  to  $x_0$
- ▶ Boundary issues with metric neighbourhood and KNN

# Popular Kernels for Smoothing

$$\hat{f}(x_0) = \frac{\sum_{i=1}^N K_\lambda(x_0, x_i) y_i}{\sum_{i=1}^N K_\lambda(x_0, x_i)} \quad K_\lambda(x_0, x) = D\left(\frac{|x - x_0|}{\lambda}\right)$$

Epanechnikov - Compact, non-differentiable at the boundary



$$D(t) = \begin{cases} \frac{3}{4}(1 - t^2), & \text{if } |t| \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

Tri-cube kernel - Compact, differentiable at the boundary

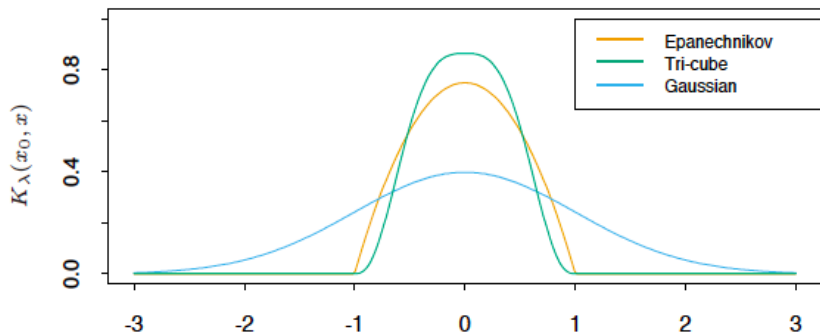


$$D(t) = \begin{cases} (1 - |t|^3)^3, & \text{if } |t| \leq 1 \\ 0, & \text{otherwise} \end{cases}$$

Gaussian kernel - non-compact, continuously differentiable

►  $D(t) = \phi(t)$

# Popular Kernels for Smoothing

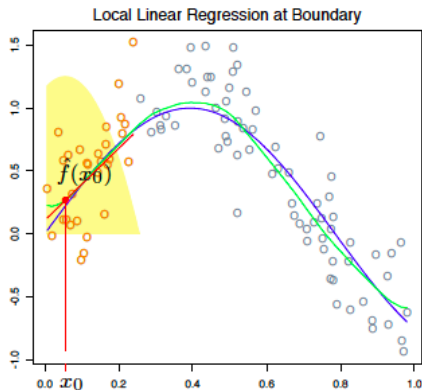
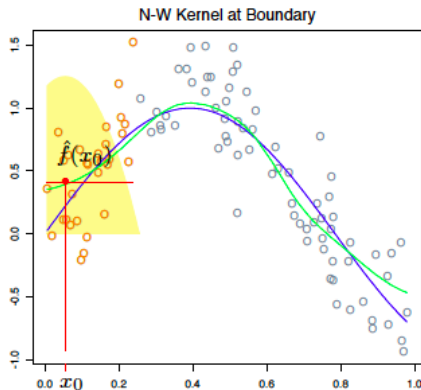


# 1-dim Kernel Smoother

## - Local Linear Regression



# Boundary Problem



Boundary problems occur due to asymmetry of the kernel

Can happen in the interior region if  $X$  is not equally spaced

# Local Linear Regression

Locally weighted regression solves a weighted least squares problem at each target point  $x_0$

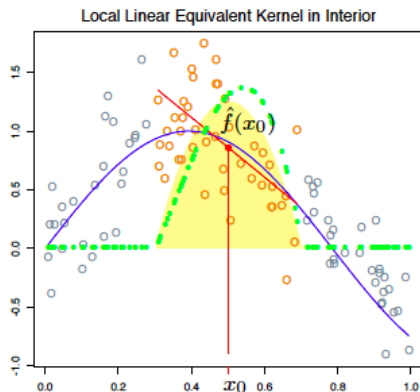
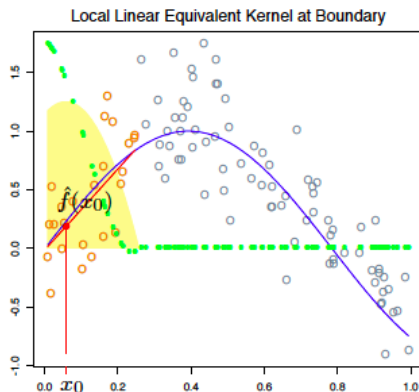
$$\min_{\alpha(x_0), \beta(x_0)} \sum_{i=1}^N K_{\lambda}(x_0, x_i) [y_i - \alpha(x_0) - \beta(x_0)x_i]^2$$

- ▶ Design matrix  $B_{N \times 2}$ , with  $i_{th}$  row  $b(x)^T = (1, x_i)$
- ▶ Weight matrix  $W_{N \times N}(x_0)$ , with  $i_{th}$  diagonal element  $K_{\lambda}(x_0, x_i)$

$$\hat{f}(x_0) = b(x_0)^T (B^T W(x_0) B)^{-1} B^T W(x_0) y = \sum_{i=1}^N l_i(x_0) y_i$$

- ▶ Estimate is linear in  $y$
- ▶  $l_i(x_0)$  combines the weighting kernel  $K_{\lambda}(x_0, \cdot)$  and least square operation, also referred to as equivalent kernel

# Bias Correction



Green:  $l_i(x_0)$  in local linear regression

Yellow:  $K_\lambda(x_0, \cdot)$  in Nadaraya-Waston

# Automatic Kernel Carpentry

Local linear regression automatically modifies the kernel to correct bias exactly to first order

$$E(\hat{f}(x_0)) = \sum_{i=1}^N l_i(x_0) f(x_i) = f(x_0) \sum_{i=1}^N l_i(x_0) + f'(x_0) \sum_{i=1}^N (x_i - x_0) l_i(x_0) + \frac{f''(x_0)}{2} \sum_{i=1}^N (x_i - x_0)^2 l_i(x_0) + R$$

It can be shown that for local linear regression

- ▶  $\sum_{i=1}^N l_i(x_0) = 1$
- ▶  $\sum_{i=1}^N (x_i - x_0) l_i(x_0) = 0$

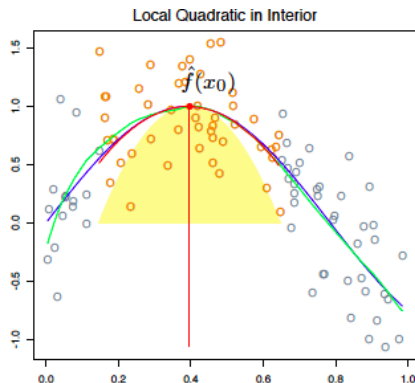
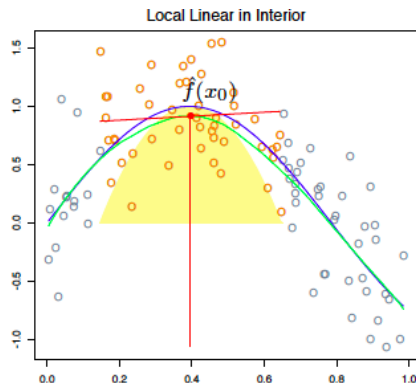
Therefore, the bias only depends on quadratic and higher order terms

- ▶  $E\hat{f}(x_0) - f(x_0) = \frac{f''(x_0)}{2} \sum_{i=1}^N (x_i - x_0)^2 l_i(x_0) + R$
- ▶ Typically small under suitable smoothness assumption

# 1-dim Kernel Smoother

- Local Polynomial Regression

# Local Polynomial Regression



Local linear fits exhibit bias in regions of curvature of the true function, while local quadratic fits tend to eliminate this bias.

# Local Polynomial Regression

Local linear fits bias: trimming the hills and filling the valleys

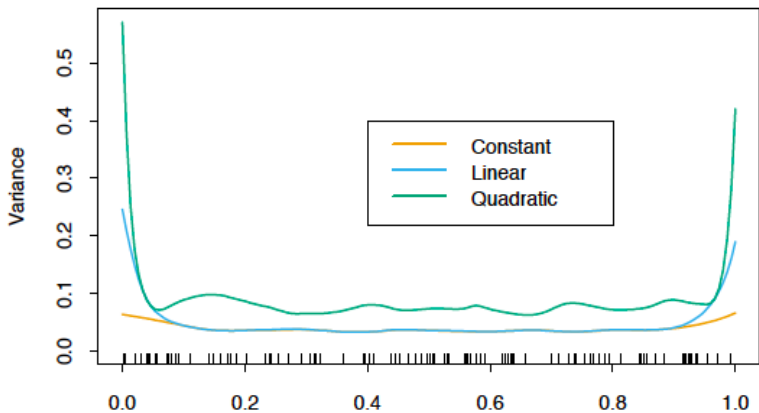
Local polynomial regression of degree-d

$$\min_{\alpha(x_0), \beta_j(x_0), j=1, \dots, d} \sum_{i=1}^N K_{\lambda}(x_0, x_i) [y_i - \alpha(x_0) - \sum_{j=1}^d \beta_j(x_0) x_i^j]^2$$

- ▶ Able to correct the bias in the regions of curvature
- ▶ But with increased variance (bias-variance tradeoff)
- ▶  $\|l(x_0)\|$  increase with d

Whether or not to choose local quadratic regression

- ▶ If interested in extrapolation (boundary)  $\rightarrow$  local linear



Variance function  $\|l(x_0)\|^2$  for local constant, linear and quadratic regression, for a metric bandwidth  $\lambda = 0.2$  tri-cube kernel.



# Local Regression in $\mathbb{R}^p$

# Local Regression in $\mathbb{R}^p$

Convert distance based kernel to radius based kernel

$$K_\lambda(x_0, x) = D\left(\frac{\|x - x_0\|}{\lambda}\right)$$

- ▶  $\|\cdot\|$  is the Euclidean norm
- ▶ Since the Euclidean norm depends on the units in each coordinate, need to standardize each  $x_j$

# Problems with local regression in $\mathbb{R}^p$

## Boundary problem

- ▶ More and more points can be found on the boundary as dimension increases
- ▶ Local polynomial regression still helps automatically deal with boundary issues for high dimensions (but not desired)

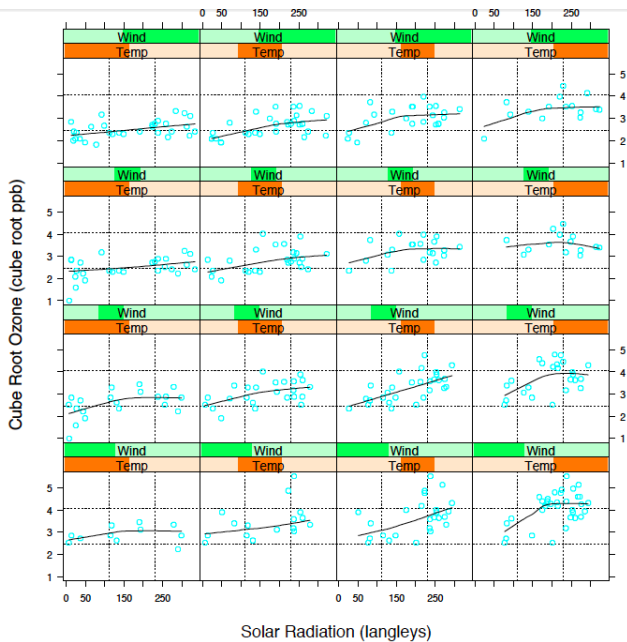
## Curse of Dimensionality

- ▶ Impossible to simultaneously maintain localness (low bias) and a sizable sample in the neighbourhood (low variance) without total sample size increasing exponentially in  $p$

## Non-visualizable

- ▶ Goal of getting a smooth fitting function is to visualize the data which is difficult in high dimensions

# Visualization



# Structured Local Regression Models in $\mathbb{R}^p$

# Structured Kernel

Spherical kernel gives equal weight to each coordinate, while a structured kernel emphasizes weight on certain coordinates using matrix  $A$

$$K_{\lambda,A}(x_0, x) = D \left( \frac{(x-x_0)^T A (x-x_0)}{\lambda} \right)$$

- ▶  $A$  is positive semidefinite
- ▶ Entire coordinates or directions can be downgraded or omitted by imposing appropriate restrictions on  $A$ .
- ▶  $A$  - diagonal?
- ▶  $A$  - low rank?
- ▶ General forms of  $A$  not recommended

# Structured Regression Functions

Suppose now we are trying to fit a regression function  $E(Y | X) = f(X_1, X_2, \dots, X_p)$  in  $\mathbb{R}^p$ , in which every level of interaction is potentially present

$$f(X_1, X_2, \dots, X_p) = \alpha + \sum_j g_j(X_j) + \sum_{k < l} g_{kl}(X_k, X_l) + \dots$$

Eliminate some higher-order terms

- ▶ Additive model (only main effects)
- ▶ Second order model (interactions up to degree 2)

Estimate low order models - Iterative backfitting algorithms

- ▶ Assume all but the  $k_{th}$  term is known
- ▶ Fit local regression of  $Y - \sum_{j \neq k} g_j(X_j)$  on  $X_k$
- ▶ Repeat for each variable until convergence

# Varying Coefficient Models

Suppose  $X \in \mathbb{R}^p$ , divide  $X$  into  $(X_1, X_2, \dots, X_q)$  with  $q < p$  and the remainder collected in vector  $Z$

$$f(X) = \alpha(Z) + \beta_1(Z)X_1 + \dots + \beta_q(Z)X_q$$

- ▶ Given  $Z$ , model is linear in  $X_1, \dots, X_q$
- ▶ Each coefficient  $\beta_1(Z), \dots, \beta_q(Z)$  vary with  $X$

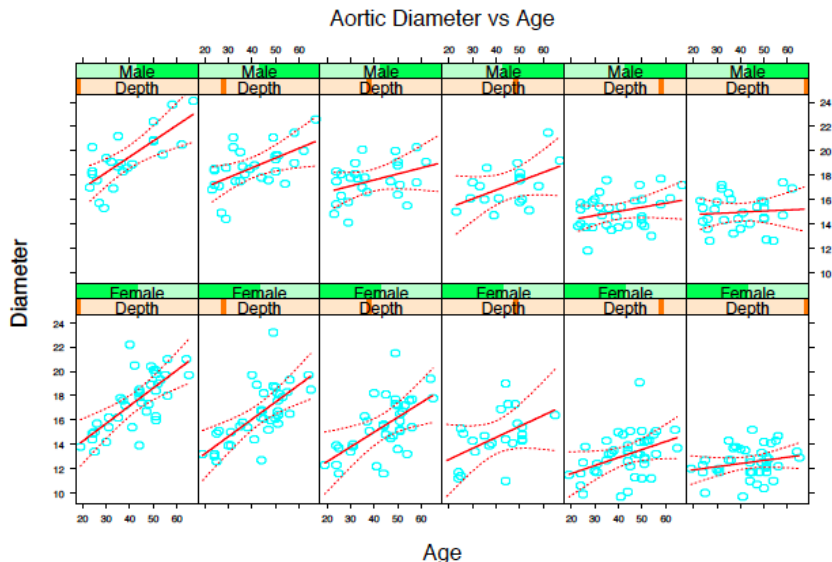
Estimate varying coefficient models - locally weighted least squares

$$\min_{\alpha(z_0), \beta(z_0)} \sum_{i=1}^N K_\lambda(z_0, z_i) (y_i - \alpha(z_0) - x_{1i}\beta_1(z_0) - \dots - x_{qi}\beta_q(z_0))^2$$



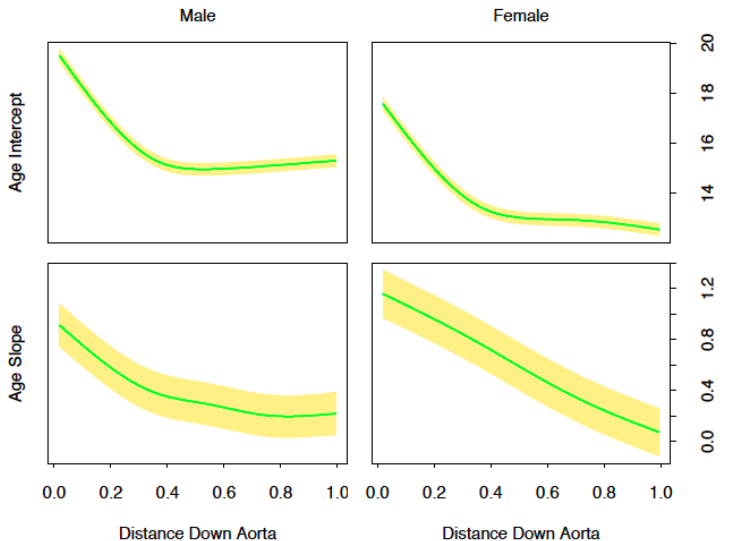
# Varying Coefficient Models

$X$  - (age, gender, depth) ,  $Y$  - aorta diameter,  $Z$  - (gender, depth)



# Varying Coefficient Models

$X$  - (age, gender, depth) ,  $Y$  - aorta diameter,  $Z$  - (gender, depth)



# Local likelihood

# Local Likelihood

Likelihood - (generalized) linear model

$$l(\beta) = \sum_{i=1}^N l(y_i, x_i^T \beta)$$

Local likelihood - relax the global linear assumption to local linear

$$l(\beta(x_0)) = \sum_{i=1}^N K_\lambda(x_0, x_i) l(y_i, x_i^T \beta(x_0))$$

## Example

Multiclass logistic regression model,  $\mathcal{G} = (1, 2, \dots, J)$

$$Pr(G = j \mid X = x) = \frac{e^{\beta_{j0} + \beta_j^\top x}}{1 + \sum_{k=1}^{J-1} e^{\beta_{k0} + \beta_k^\top x}}$$

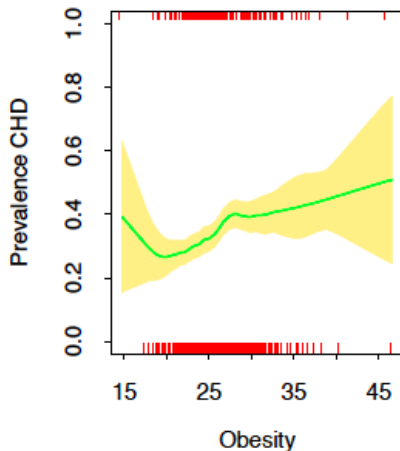
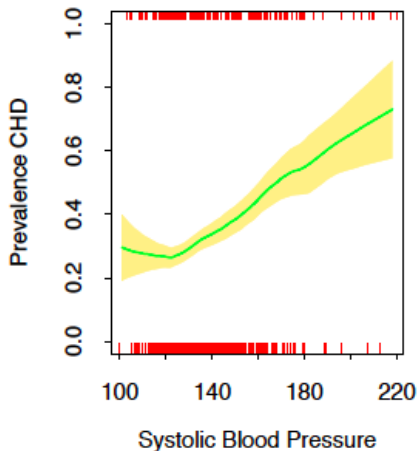
The local version of multiclass logistic regression

$$\begin{aligned} l(\beta(x_0)) = & \sum_{i=1}^N K_\lambda(x_0, x_i) \{ \beta_{g_i,0}(x_0) + \beta_{g_i}(x_0)^\top (x_i - x_0) \\ & - \log[1 + \sum_{k=1}^{J-1} \exp(\beta_{k,0}(x_0) + \beta_k(x_0)^\top (x_i - x_0))] \} \end{aligned}$$

The fitted posterior probabilities

$$Pr(G = j \mid X = x_0) = \frac{e^{\hat{\beta}_{j0}(x_0)}}{1 + \sum_{k=1}^{J-1} e^{\hat{\beta}_{k0}(x_0)}}$$

# Example: South African Heart Disease Data



# Kernel Density Estimation & Classification

# Kernel Density Estimation

Suppose we've got a random sample  $x_1, \dots, x_N$  drawn from  $f_X(x)$ .

How do you estimate  $f_X$  at a point  $x_0$ ?



# Kernel Density Estimation

Empirical local estimate - bumpy

$$\hat{f}_X(x_0) = \frac{\#x_i \in \mathcal{N}(x_0)}{N_\lambda}$$

- $\mathcal{N}(x_0)$  - small metric neighbourhood around  $x_0$  of width  $\lambda$

Parzen estimate - smooth

$$\hat{f}_X(x_0) = \frac{1}{N_\lambda} \sum_{i=1}^N K_\lambda(x_0, x_i)$$

- Counts observations with weights that decrease with distance
- Choice of  $K_\lambda$ : Gaussian Kernel  $K_\lambda(x_0, x) = \phi\left(\frac{|x-x_0|}{\lambda}\right)$

# Kernel Density Estimation

Gaussian kernel density estimate

$$\hat{f}_X(x) = \frac{1}{N} \sum_{i=1}^N \phi_\lambda(x - x_i) = (\hat{F} * \phi_\lambda)(x)$$

- ▶  $\phi_\lambda$  denotes Gaussian density with mean 0 and standard deviation  $\lambda$
- ▶ Convolution of the sample empirical distribution  $\hat{F}$  (jumpy) with  $\phi_\lambda$  (smooth)

Gaussian kernel density estimate in  $\mathbb{R}^p$

$$\hat{f}_X(x_0) = \frac{1}{N(2\lambda^2\pi)^{\frac{p}{2}}} \sum_{i=1}^N e^{-\frac{1}{2}(\|x_i - x_0\|/\lambda)^2}$$

# Kernel Density Classification

For a  $J$  class problem, suppose we can fit nonparametric density estimates  $\hat{f}_j(X)$  separately for each class, and we also have estimates of the class priors  $\hat{\pi}_j$

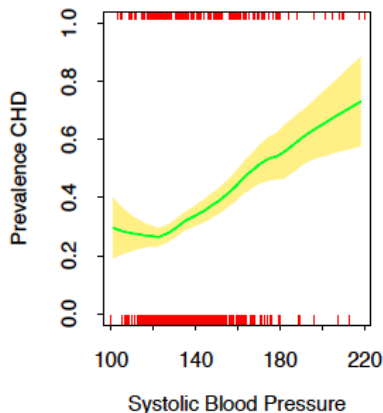
Then by Bayes Theorem, we can derive the class posterior probabilities

$$\hat{Pr}(G = j \mid X = x_0) = \frac{\hat{\pi}_j \hat{f}_j(x_0)}{\sum_{k=1}^J \hat{\pi}_k \hat{f}_k(x_0)}$$

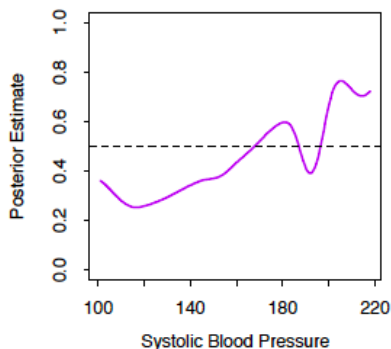
If classification is the ultimate goal

- ▶ Learning the separate class densities well may be unnecessary or misleading
- ▶ Need only estimate the posterior well near the decision boundary

# Kernel Density Classification



- ▶ Local logistic method estimates
- ▶ Use tri-cube kernel with k-NN bandwidth



- ▶ Gaussian kernel density estimates
- ▶ Data are sparse in high SBP region

# Naive Bayes Classifier

Assumes that given class  $G = j$ , the features  $X_k$  are independent

$$f_j(X) = \prod_{k=1}^p f_{jk}(X_k)$$

- ▶ Useful when  $p$  is large, when density estimation is unattractive
- ▶ Each  $f_{jk}$  can be estimated using 1-dim kernel density estimates
- ▶ If  $X_j$  is discrete, can be estimated by appropriate histogram
- ▶ Sometimes outperform far more sophisticated models

$$\begin{aligned}\log \frac{Pr(G = I \mid X)}{Pr(G = J \mid X)} &= \log \frac{\pi_I f_I(X)}{\pi_J f_J(X)} = \log \frac{\pi_I \prod_{k=1}^p f_{Ik}(X_k)}{\pi_J \prod_{k=1}^p f_{Jk}(X_k)} \\ &= \log \frac{\pi_I}{\pi_J} + \sum_{k=1}^p \log \frac{f_{Ik}(X_k)}{f_{Jk}(X_k)} = \alpha_I + \sum_{k=1}^p g_{Ik}(X_k)\end{aligned}$$

# Radial Basis Function & Kernels

# Radial Basis Function

$$f(x) = \sum_{j=1}^M K_{\lambda_j}(\xi_j, x) \beta_j = \sum_{j=1}^M D\left(\frac{\|x - \xi_j\|}{\lambda_j}\right) \beta_j$$

- ▶ Treat the kernel function  $K_{\lambda}(\xi, x)$  as basis functions
- ▶ Each basis element is indexed by a location parameter  $\xi_j$  and a scale parameter  $\lambda_j$
- ▶ Popular choice for  $D$  - standard Gaussian

Approaches for learning parameters  $\{\lambda_j, \xi_j, \beta_j\}$

- ▶  $\min_{\{\lambda_j, \xi_j, \beta_j\}_1^M} \sum_{i=1}^N \left( y_i - \beta_0 - \sum_{j=1}^M \beta_j \exp \left\{ -\frac{(x_i - \xi_j)^T (x_i - \xi_j)}{\lambda_j^2} \right\} \right)^2$
- ▶ Estimate  $\{\lambda_j, \xi_j\}$  (often unsupervised) separately from  $\beta_j$

# Mixture Models for Density Estimation & Classification



# Gaussian Mixture

$$f(x) = \sum_{m=1}^M \alpha_m \phi(x; \mu_m, \Sigma_m)$$

- ▶ Mixing proportions  $\alpha_m$  with  $\sum_m \alpha_m = 1$
- ▶ Mean  $\mu_m$ , covariance matrix  $\Sigma_m$
- ▶ Usually fit by maximum likelihood using EM algorithm

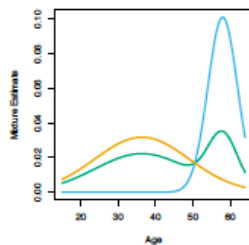
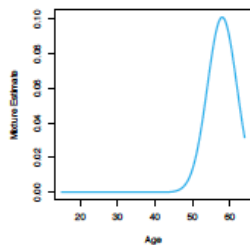
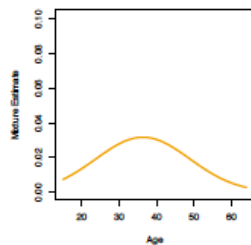
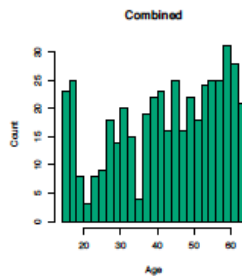
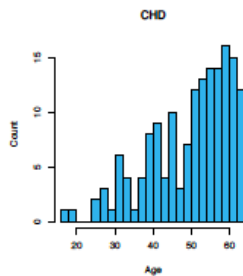
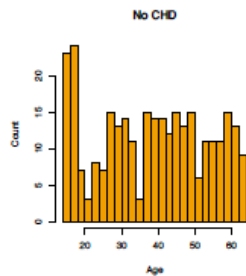
## Special cases

- ▶ If  $\Sigma_m = \sigma_m I$ ,  $f(x)$  has the form of a radial basis expansion
- ▶ If in addition  $\sigma_m = \sigma > 0$  is fixed and  $M \uparrow N$ , the MLE approaches the kernel density estimate where  $\hat{\alpha}_m = \frac{1}{N}$  and  $\hat{\mu}_m = x_m$

$$\hat{r}_{im} = \frac{\hat{\alpha}_m \phi(x_i; \hat{\mu}_m, \hat{\Sigma}_m)}{\sum_{k=1}^M \hat{\alpha}_k \phi(x_i; \hat{\mu}_k, \hat{\Sigma}_k)}$$

- ▶ Estimate of the probability that the  $i_{th}$  observation belongs to component  $m$

# Gaussian Mixture



# Gaussian Mixture for Classification

$X$  - age,  $Y$  - CHD / no CHD

$\hat{r}_{i1}$  estimates the probability that the  $i_{th}$  person does not have CHD, while  $\hat{r}_{i2}$  estimates the probability that the  $i_{th}$  person has CHD.

Using threshold of 0.5, i.e  $\delta_i = I(\hat{r}_{i2} > 0.5)$ , compare the classification with the data

|     |     | Mixture model      |                    |
|-----|-----|--------------------|--------------------|
|     |     | $\hat{\delta} = 0$ | $\hat{\delta} = 1$ |
| CHD | No  | 232                | 70                 |
|     | Yes | 76                 | 84                 |